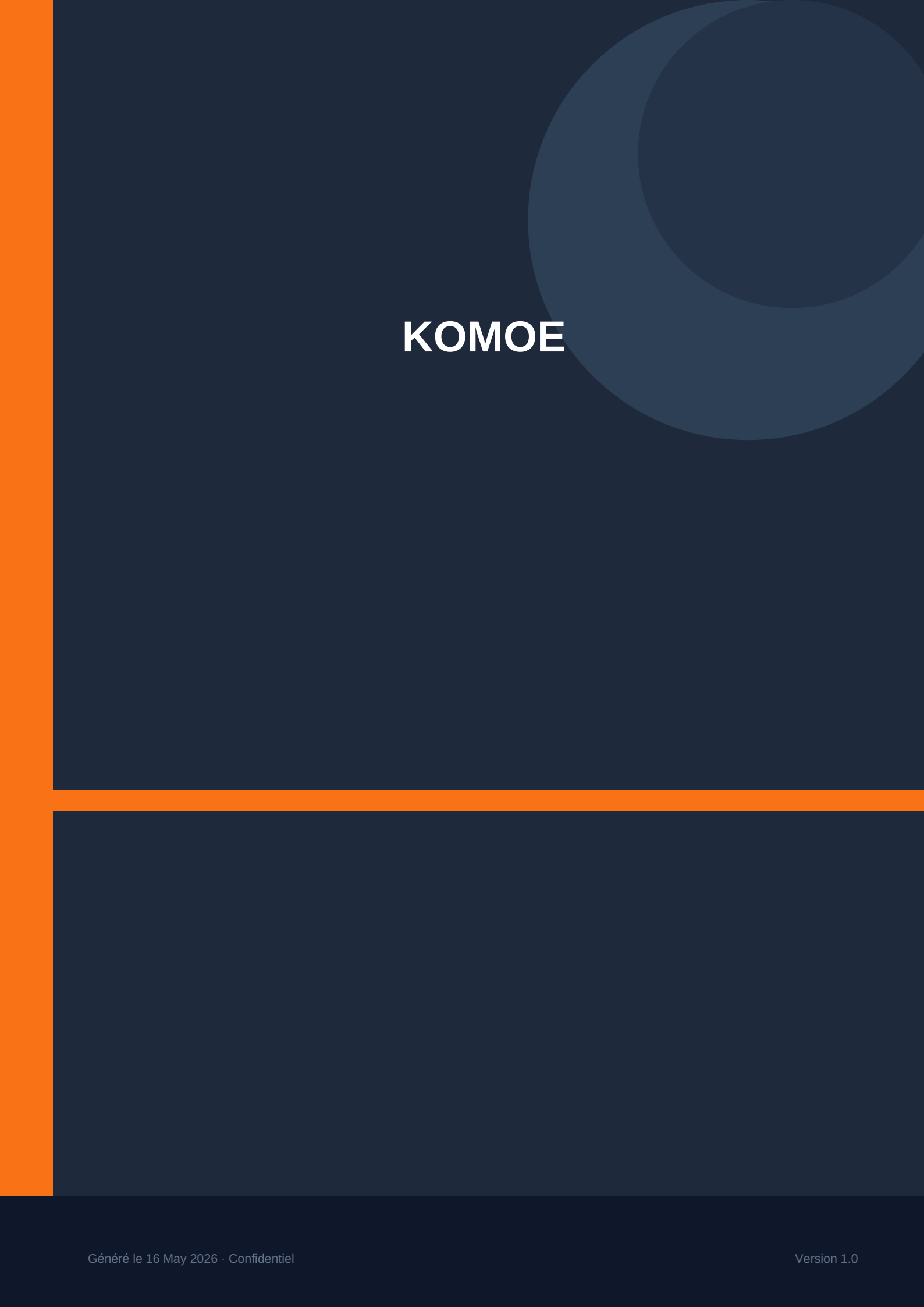




KOMOE

Documentation Technique Complète

Plateforme de Transparence Budgétaire des Communes Ivoiriennes

Django 5.1

Next.js 16

Polygon / Solidity

Gemini AI

IPFS / Pinata

Table des Matières

1. Vue d'ensemble du projet
2. Architecture technique générale
3. Backend Django — Structure et configuration
 - 3.1 Modèle utilisateur & Authentification
 - 3.2 Module Communes & Scoring
 - 3.3 Module Transactions Financières
 - 3.4 Module Signalements & Budget Participatif
 - 3.5 Service Blockchain (Web3.py)
4. Smart Contract BudgetLedger.sol
 - 4.1 Rôles et contrôle d'accès
 - 4.2 Fonctions principales
 - 4.3 Events — Registre immuable
5. Frontend Next.js — Structure et pages
 - 5.1 Espaces utilisateurs et navigation
 - 5.2 Bibliothèque API (lib/api.ts)
 - 5.3 MIA Bot — Intelligence Artificielle
 - 5.4 Intégration blockchain (wagmi / RainbowKit)
6. API REST — Référence complète
 - 6.1 Endpoints Auth & Utilisateurs
 - 6.2 Endpoints Communes & Projets
 - 6.3 Endpoints Transactions
 - 6.4 Endpoints Signalements & Propositions
7. Modèles de données
8. Flux de données principaux
9. Sécurité
10. Déploiement

1. Vue d'ensemble du projet

KOMOE est une plateforme de transparence et de gouvernance budgétaire des communes ivoiriennes, développée dans le cadre de l'initiative **MIABE 2026** portée par la **Direction Générale de la Décentralisation et du Développement Local (DGDDL)** du Ministère de l'Intérieur de Côte d'Ivoire. Elle couvre les **201 communes** du territoire national.

Objectifs principaux

- **Transparence financière** : Rendre publics les budgets, dépenses et recettes communales en temps réel.
- **Traçabilité blockchain** : Ancrer chaque transaction validée sur la blockchain Polygon pour garantir l'immutabilité.
- **Démocratie participative** : Permettre aux citoyens de proposer, voter et suivre les projets de leur commune.
- **Lutte contre la corruption** : Système de signalement citoyen avec enquête DGDDL et verdict ancré on-chain.
- **Open Data** : Exposer les données publiques en CSV et via API ouverte pour chercheurs et bailleurs.

Les 7 acteurs du système

AGENT_FINANCIER	Saisit les dépenses/recettes (brouillons), soumet à la validation du Maire	/commune
MAIRE	Valide et signe les transactions sur la blockchain, gère le Budget Participatif	/commune
DGDDL	Super-admin national : gère les 201 communes, attribue les rôles, lance les enquêtes	/controle
COUR_COMPTES	Auditeur financier national : accès lecture complet toutes communes	/controle
BAILLEUR	Bailleur international : suit les projets financés, score de transparence	/public
CITOYEN	Lecture des données publiques de sa commune, vote, signalement	/public
JOURNALISTE	Export multi-communes, Open Data, signalements — (fusionné avec CITOYEN+profession)	/public

2. Architecture Technique Générale

KOMOE adopte une architecture **microservices découplée** en trois couches distinctes communiquant via des interfaces standardisées (REST API + Events blockchain).

Couche	Technologie	Rôle	Déploiement
Frontend	Next.js 16 + React 19 TypeScript + Tailwind CSS	Interface utilisateur SPA avec 4 espaces différenciés par rôle	Vercel (CDN mondial)
Backend	Django 5.1 + DRF Python 3.13	API REST JWT, logique métier, orchestration blockchain	Render (Gunicorn + WhiteNoise)
Blockchain	Solidity 0.8.20 Polygon Amoy (Chain ID 80002)	Registre immuable des transactions et enquêtes	Polygon Network (testnet)
Stockage IPFS	Pinata Gateway	Archivage décentralisé des justificatifs PDF	IPFS / Pinata Cloud
Base de données	PostgreSQL (prod) SQLite (dev)	Persistance relationnelle des données applicatives	Render Postgres
IA embarquée	Google Gemini Flash (Function Calling)	MIA Bot — assistant vocal contextuel par rôle	Google Cloud AI

Flux de communication

Le frontend communique avec le backend exclusivement via HTTPS REST (JSON). Le backend orchestre les interactions blockchain via Web3.py en mode **custodial** : la clé privée du déployeur DGDDL est stockée côté serveur (variable d'environnement) et signe les transactions à la demande des rôles autorisés. Le frontend peut également interagir directement avec la blockchain via MetaMask/RainbowKit pour la lecture publique des events.

Architecture custodiale (MVP) : En phase de production avancée, la signature devrait migrer vers un modèle non-custodial (MetaMask côté client) pour chaque Maire/Agent. Le mode custodial actuel simplifie l'onboarding des acteurs institutionnels peu familiers du Web3.

3. Backend Django — Structure et Configuration

Le backend est un projet Django 5.1 organisé en applications modulaires, exposant une API REST via Django REST Framework avec authentification JWT (SimpleJWT).

Structure des applications

Application	Chemin	Responsabilité
apps.users	backend/apps/users/	Utilisateurs, JWT, certifications, professions
apps.communes	backend/apps/communes/	Communes, projets, scoring de transparence
apps.transactions	backend/apps/transactions/	Transactions, signalements, propositions, votes, notifications
apps.blockchain	backend/apps/blockchain/	Service Web3.py — interaction avec BudgetLedger.sol
config	backend/config/	Settings, URLs, WSGI

Configuration clé (settings.py)

Toutes les valeurs sensibles sont chargées depuis des variables d'environnement via python-dotenv. La base de données est PostgreSQL en production et SQLite en développement local (fallback automatique). Le fuseau horaire est configuré sur **Africa/Abidjan** et la langue sur **fr-fr**.

- **AUTH_USER_MODEL** : apps.users.User — modèle utilisateur entièrement personnalisé
- **JWT Access Token** : 60 minutes (configurable via JWT_ACCESS_TOKEN_LIFETIME_MINUTES)
- **JWT Refresh Token** : 7 jours avec rotation automatique
- **CORS** : Origines autorisées via CORS_ALLOWED_ORIGINS (env)
- **Logging** : Fichier logs/backend.log + console, niveau INFO
- **Static files** : WhiteNoise (CompressedManifestStaticFilesStorage)

3.1 Modèle Utilisateur & Authentification

Le modèle **User** étend AbstractBaseUser avec une authentification par email (pas de username). Chaque utilisateur possède un rôle unique parmi les 7 définis.

Champs principaux du modèle User

Champ	Type	Description
id	UUIDField (PK)	Identifiant UUID unique, auto-généré
email	EmailField (unique)	Identifiant de connexion
nom / prenom	CharField	Identité civile
role	CharField (choices)	Rôle fonctionnel (7 valeurs)

commune	ForeignKey Commune	→ Commune d'affectation (obligatoire pour Agent/Maire)
wallet_address	CharField(42)	Adresse Ethereum (Polygon) — mode custodial
profession	CharField (choices)	Profession publique : CITOYEN, JOURNALISTE, ONG, CHERCHEUR, BAILLEUR
professions	JSONField	Liste de professions multiples (citoyens)
certification_status	CharField	Statut Sentinelle : NONE / PENDING / APPROVED / REJECTED
is_blockchain_authorized	BooleanField	Autorisé à effectuer des opérations blockchain
reputation_score	IntegerField	Score de réputation citoyen (0+)
journaliste_verifie	BooleanField	Journaliste vérifié par DGDDL
profession_verified	BooleanField	Profession vérifiée avec document

Système de Certification Sentinelle

La certification Sentinelle est un processus de vérification d'identité en 3 étapes permettant aux citoyens de gagner la confiance de la plateforme pour voter sur des signalements et propositions :

1. Le citoyen soumet son numéro CNI + date d'expiration + document scan.
2. La DGDDL (ou le Maire local) examine la demande via l'endpoint /certification/pending/.
3. Après approbation, le champ is_blockchain_authorized passe à True, autorisant les actions sensibles.

Permissions définies (users/permissions.py)

- `IsDGDDL` : Réservé au rôle DGDDL uniquement
- `IsMaire` : Réservé au rôle MAIRE
- `IsAgentFinancier` : Réservé au rôle AGENT_FINANCIER
- `IsCourComptes` : Réservé à la Cour des Comptes
- `IsMaireOfCommune` : Vérifie que le Maire appartient à la commune de l'objet cible
- `IsVerifiedUser` : Citoyen certifié Sentinelle OU acteur institutionnel rattaché
- `IsJournalisteVerified` : Journaliste avec profession vérifiée par document

3.2 Module Communes & Scoring

Modèle Commune

Champ	Type	Description
code	CharField(20, unique)	Code INSEE ou code interne de la commune
nom	CharField(200)	Nom officiel de la commune
region	CharField (choices)	Région parmi : Abidjan, Bouaké, Yamoussoukro, Daloa, San-Pédro, Korhogo, Autre
population	PositiveIntegerField	Population totale
budget_annuel_fcfa	BigIntegerField	Dotation annuelle de la DGDDL en FCFA
score_transparence	PositiveIntegerField	Score composite calculé dynamiquement (0-100)
maire_nom	CharField	Nom du Maire en exercice
blockchain_tx_hash_dotation	CharField	Hash de la transaction de dotation on-chain

Modèle Projet

Champ	Type	Description
commune	ForeignKey Commune →	Commune portant le projet
nom	CharField(255)	Intitulé du projet
budget_alloue_fcfa	BigIntegerField	Budget initial alloué
budget_consomme_fcfa	BigIntegerField	Budget réellement dépensé
taux_execution	FloatField (0-100)	Taux d'avancement physique
statut	CharField (choices)	BROUILLON / EN_ATTENTE / EN_COURS / ACHEVE / ANNULE / SOUS_ENQUETE
bailleur	FK → User (BAILLEUR)	Bailleur international finançant ce projet
parent_proposition	FK → PropositionDepense	Idee citoyenne à l'origine du projet (Budget Participatif)

Score de Transparence — Algorithme composite

Le score de transparence (0-100) est calculé dynamiquement à chaque requête via la fonction **calculer_score_composite(commune)** selon 5 dimensions pondérées :

Dimension	Poids	Calcul
Taux de validation	30%	Transactions validées / Total soumises x 100
Documentation IPFS	25%	Transactions avec hash IPFS / Total validées x 100
Réactivité	20%	Score fixe 85 si validations existent, sinon 0
Engagement citoyen	15%	$\min(\text{nb_propositions} \times 10, 100)$
Intégrité	10%	$\max(0, 100 - \text{ratio_signalements} \times 500)$

3.3 Module Transactions Financières

Le modèle Transaction est le coeur du système financier. Il représente toute opération budgétaire (dépense ou recette) et suit un cycle de vie en plusieurs statuts.

Cycle de vie d'une transaction

- BROUILLON : Créé par l'Agent Financier. Visible uniquement en interne.
- SOUMIS : Soumis sur la blockchain par l'Agent (event DepenseSoumise). En attente de validation Maire.
- VALIDE : Validé par le Maire via validerDepense() on-chain. Preuve publique immuable disponible.
- REJETE : Rejeté par le Maire avec motif. L'Agent peut corriger et re-soumettre.
- FRAUDULEUSE : Marquée comme frauduleuse suite à une enquête DGDDL confirmée.
- CORRIGEE : Version corrigée d'une transaction frauduleuse (parent_frauduleux renseigné).

Champs blockchain d'une Transaction

Champ	Type	Description
blockchain_tx_hash_soumission	CharField(100)	Hash Polygon de la soumission Agent
blockchain_tx_hash_validation	CharField(100)	Hash Polygon de la validation Maire
blockchain_synced_at	DateTimeField	Date de synchronisation blockchain
ipfs_hash	CharField(100)	Hash IPFS du justificatif PDF
ipfs_url	URLField	URL publique Pinata Gateway du document
is_correction	BooleanField	True si correction d'une transaction frauduleuse
parent_frauduleux	FK → Transaction	Référence vers la transaction originale annulée

Catégories de dépenses (alignées ODD)

INFRASTRUCTURE (ODD 9), SANTE (ODD 3), EDUCATION (ODD 4), EAU_ASSAINISSEMENT (ODD 6), SECURITE (ODD 16), ADMINISTRATION (ODD 16), AGRICULTURE (ODD 2), CULTURE_SPORT, ENVIRONNEMENT, SOCIAL, AUTRE

3.4 Module Signalements & Budget Participatif

Système de Signalement Citoyen

Les citoyens peuvent signaler des anomalies financières. Chaque signalement suit un workflow complet de l'alerte jusqu'à la résolution publique ancrée on-chain :

- **NOUVEAU** : Signalement créé, pas encore en vote.
- **ACTIF** : En cours de vote citoyen (CREDIBLE / INFONDE).
- **VIRAL** : Seuil de 4 votes atteint — alerte DGDDL déclenchée automatiquement.
- **ENQUETE_DGDDL** : DGDDL a officiellement lancé l'enquête (ancré on-chain via `lancerEnquete()`).
- **VALIDE_FRAUDE** : Fraude confirmée par DGDDL — transaction mère marquée FRAUDULEUSE.
- **REJETE_FAUX** : Signalement infondé après enquête.
- **CLOS** : Dossier archivé.

Budget Participatif (PropositionDepense)

Les citoyens peuvent soumettre des idées de projets. Le workflow de gouvernance est le suivant :

1. Le citoyen crée une proposition (titre, description, `budget_demande_fcfa`, catégorie).
2. Les citoyens votent SOUTIEN ou OPPOSITION pendant la période de vote.
3. Le Maire officialise la proposition (`officialiserProposition()` on-chain) si elle est retenue.
4. Après le vote officiel, le Maire clôture (`cloturerProposition()` on-chain) — résultat immuable.
5. Si approuvée, la proposition est convertie en Projet officiel (statut CONVERTIE).

Autres modèles du module transactions

- **VoteSignalement** : Vote citoyen sur un signalement (CREDIBLE ou INFONDE) — unique par utilisateur/signalement
- **VoteProposition** : Vote SOUTIEN/OPPOSITION sur une proposition — unique par citoyen/proposition
- **CommentaireSignalement** : Commentaire avec type AVIS, JUSTIFICATION (Maire) ou ENQUETE (DGDDL)
- **CommentaireProposition** : Commentaire sur une proposition (AVIS ou NOTE_TECHNIQUE)
- **PreuveSignalement** : Fichier IPFS joint à un signalement (image, PDF, autre)
- **PreuveProposition** : Fichier IPFS joint à une proposition
- **ActionDGDDL** : Journal des actions d'investigation DGDDL sur un signalement
- **Notification** : Notifications in-app par SSE (Server-Sent Events) — types: TRANSACTION, VOTE, SIGNALEMENT...
- **RapportPDF** : Archive des rapports de transparence mensuels générés et hébergés sur IPFS
- **VoteProjet** : Soutien/Like sur un projet officiel en cours
- **CommentaireProjet** : Retour terrain citoyen sur un projet (avec photo IPFS)

3.5 Service Blockchain — BlockchainService (Web3.py)

La classe BlockchainService encapsule toutes les interactions avec le smart contract BudgetLedger.sol via Web3.py. Elle utilise un pattern de lazy loading pour les connexions et signe toutes les transactions côté serveur (mode custodial).

Méthodes principales

- `soumettre_depense()` : Appelle `soumettreDepense()` — AGENT_ROLE
- `valider_depense()` : Appelle `validerDepense()` — MAIRE_ROLE (anti double-validation)
- `soumettre_recette()` : Appelle `soumettreRecette()` — AGENT_ROLE
- `enregistrer_recette()` : Appelle `enregistrerRecette()` — MAIRE_ROLE
- `officialiser_proposition()` : Ancre l'officialisation d'une proposition citoyenne
- `cloturer_proposition()` : Ancre le résultat du vote citoyen (approuvée + nb votes)
- `lancer_enquete()` : DGDDL lance officiellement une enquête — EnqueteLancee event
- `resoudre_enquete()` : DGDDL ancre le verdict final (FRAUDE / FAUX / INFONDE)
- `attribuer_role_agent()` : Attribue AGENT_ROLE on-chain pour une commune
- `attribuer_role_maire()` : Attribue MAIRE_ROLE on-chain pour une commune
- `enregistrer_dotation()` : DGDDL ancre une dotation budgétaire on-chain
- `verifier_hash_transaction()` : Vérifie l'existence et la validité d'un TX hash on-chain
- `pause()` / `unpause()` : Pause d'urgence du contrat (DEFAULT_ADMIN_ROLE)

Optimisation du gas : La méthode `_send_transaction()` estime automatiquement le gas ($\text{estimate_gas} \times 1.1 = \text{buffer de 10\%}$) avant chaque transaction. En cas d'échec de l'estimation, un fallback de 500 000 gas est utilisé. Le nonce est calculé dynamiquement pour chaque transaction.

4. Smart Contract BudgetLedger.sol

Le contrat **BudgetLedger** est déployé sur le réseau **Polygon Amoy Testnet** (Chain ID 80002). Il est écrit en Solidity 0.8.20 et hérite des contrats OpenZeppelin AccessControl et Pausable pour la gestion des rôles et la pause d'urgence.

Réseau	Polygon Amoy Testnet (Chain ID : 80002)
Adresse	0xDd60F74Dbca514C8eD1AfecFad054F416B18E6D4
Explorateur	https://amoy.polygonscan.com
Compilateur	Solidity 0.8.20
Standards	OpenZeppelin AccessControl + Pausable
ABI	lib/abi/BudgetLedger.json (frontend) + apps/blockchain/service.py (backend)

4.1 Rôles et contrôle d'accès

Le contrat utilise le système de rôles d'OpenZeppelin AccessControl avec 3 rôles hiérarchiques :

- `DEFAULT_ADMIN_ROLE` : DGDDL — déployeur initial. Peut attribuer/révoquer tous les rôles, lancer des enquêtes, enregistrer des dotations, pauser le contrat.
- `AGENT_ROLE` : Agent Financier communal. Peut soumettre des dépenses et recettes (brouillons blockchain). Scopé à une commune via le mapping `walletCommune`.
- `MAIRE_ROLE` : Maire communal. Peut valider définitivement les dépenses/recettes, officialiser et clôturer les propositions. Scopé à sa commune via `walletCommune`.

■ **Note** : Le mapping `walletCommune[address]` => `communeId` scope chaque acteur à une seule commune. Un agent/maire ne peut soumettre/valider que pour sa commune. L'admin DGDDL est exempt de cette restriction (mode 'pour toutes les communes').

4.2 Fonctions principales

Fonction	Rôle requis	Description
<code>soumettreDepense(depenseId, communeId, montant, categorie, ipfsHash)</code>	<code>AGENT_ROLE</code>	Soumettre une dépense — émet <code>DepenseSoumise</code>
<code>validerDepense(depenseId, communeId, montant, categorie, ipfsHash)</code>	<code>MAIRE_ROLE</code>	Valider définitivement — anti double-validation par mapping <code>bytes32</code>
<code>soumettreRecette(recetteId, communeId, montant, source, ipfsHash)</code>	<code>AGENT_ROLE</code>	Soumettre une recette
<code>enregistrerRecette(recetteId, communeId, montant, source, ipfsHash)</code>	<code>MAIRE_ROLE</code>	Officialiser/valider une recette
<code>officialiserProposition(propositionId, communeId, ipfsHash)</code>	<code>MAIRE_ROLE</code>	Engage officiellement une proposition citoyenne
<code>cloturerProposition(propositionId, communeId, ipfsHash)</code>	<code>MAIRE_ROLE</code>	Accrédite le résultat d'une proposition
<code>lancerEnquete(signalementId, communeId)</code>	ADMIN	DGDDL lance une enquête officielle
<code>resoudreEnquete(signalementId, communeId, decision)</code>	ADMIN	DGDDL ancre le verdict

<code>enregistrerDotation(communeId, montant)</code>	ADMIN	Enregistre une dotation budgétaire
<code>attribuerRoleAgent(wallet, communeId)</code>	ADMIN	Donne AGENT_ROLE + scope commune
<code>attribuerRoleMaire(wallet, communeId)</code>	ADMIN	Donne MAIRE_ROLE + scope commune
<code>revoquerRole(role, wallet)</code>	ADMIN	Révoque un rôle et supprime le scope commune
<code>pause() / unpause()</code>	ADMIN	Circuit-breaker d'urgence

4.3 Events — Registre immuable public

Les events Solidity constituent le registre public et immuable de toutes les opérations. Ils sont indexés pour permettre une lecture efficace via Ethers.js ou Web3.py. Chaque event horodaté avec `block.timestamp` constitue une preuve cryptographique vérifiable par n'importe quel auditeur indépendant.

Event	Émetteur	Paramètres (indexés marqués *)
DepenseSoumise	AGENT	depenseId*, communeId*, montant, categorie, ipfsHash, soumisePar*, timestamp
DepenseValidee	MAIRE	depenseId*, communeId*, montant, categorie, ipfsHash, validePar*, timestamp
RecetteSoumise	AGENT	recetteId*, communeId*, montant, source, ipfsHash, parAgent*, timestamp
RecetteEnregistree	MAIRE	recetteId*, communeId*, montant, source, ipfsHash, enregistreePar*, timestamp
DotationEnregistree	ADMIN	communeId*, montant, timestamp
PropositionOfficialisee	MAIRE	propositionId*, communeId*, ipfsHash, parMaire*, timestamp
PropositionCloturee	MAIRE	propositionId*, communeId*, approuvee, nbVotesSoutien, nbVotesOpposition, timestamp
EnqueteLancee	ADMIN	signalementId*, communeId*, parDGDDL*, timestamp
EnqueteResolue	ADMIN	signalementId*, communeId*, resolution, parDGDDL*, timestamp
AgentRoleAttribue	ADMIN	wallet*, parAdmin*, timestamp
MaireRoleAttribue	ADMIN	wallet*, parAdmin*, timestamp
RoleRevoque	ADMIN	role*, wallet*, parAdmin*, timestamp

5. Frontend Next.js — Structure et Pages

Le frontend est une application Next.js 16 (App Router) en TypeScript avec Tailwind CSS. Il expose **4 espaces utilisateurs distincts** avec navigation et permissions adaptées à chaque rôle.

5.1 Espaces utilisateurs et navigation

Espace	URL de base	Rôles autorisés	Pages principales
Mairie	/commune	AGENT_FINANCIER, MAIRE	Dashboard, Saisies, Projets, Budget Participatif, Signalements, B
Contrôle	/controle	DGDDL, COUR_COMPTES	Vue nationale, Dotations, Comptes, Certification, Communes, Cla
Public	/public	CITOYEN, JOURNALISTE, BAILLEUR	Dashboard, Engagements, Projets, Transactions, Budget, Signale
Bailleur	/bailleur	BAILLEUR	Redirect vers /public (espace commun)

Composants UI principaux

- `AppLayout` : Layout global avec Sidebar, Header et MiaBot intégrés
- `Sidebar` : Navigation contextuelle générée dynamiquement selon le rôle — collapsable, mobile-friendly
- `Header` : Barre supérieure avec thème, notifications (`NotificationBell`), profil
- `NotificationBell` : Cloche temps réel via SSE (Server-Sent Events) — badge de comptage
- `DataTable` : Tableau de données réutilisable avec tri, filtrage, pagination
- `ReusableForm` : Formulaire générique configurable par schema
- `BlockchainMap` : Visualisation réseau force-graph (react-force-graph-2d) des transactions
- `StatsCard` : Carte de statistique avec icône, valeur et variation
- `SectionCommentaires` : Section commentaires avec types AVIS / JUSTIFICATION / ENQUETE
- `DocumentPreview` : Prévisualisation de fichier IPFS (PDF, image)
- `SignalementVoter` : Interface de vote CREDIBLE/INFONDE sur un signalement
- `ProjetCard` : Carte visuelle d'un projet avec taux d'exécution

5.2 Bibliothèque API (lib/api.ts)

Le module `api.ts` centralise tous les appels HTTP. Il implémente un wrapper `apiFetch()` avec gestion automatique du JWT (refresh token si 401), stockage des tokens dans `localStorage` et cookie (pour le middleware `proxy.ts`). Chaque domaine possède son namespace d'API :

- `authApi` : Login, register, me, users CRUD, blockchain autorisation, certification Sentinelle
- `communesApi` : Liste, détail communes, confirmation dotation
- `transactionsApi` : Liste, soumission, validation, rejet des transactions
- `signalementsApi` : CRUD signalements, votes, commentaires, preuves, enquêtes DGDDL
- `propositionsApi` : CRUD propositions, votes, officialisation, clôture Maire
- `projetsApi` : Liste, détail, mise à jour, suppression des projets
- `notificationsApi` : Liste notifications, marquage lu, URL SSE stream
- `rapportsApi` : URLs de téléchargement PDF et export CSV

- `anomaliesApi` : Détection d'anomalies
- `openDataApi` : Statistiques open data publiques

5.3 MIA Bot — Intelligence Artificielle Intégrée

MIA (Municipal Intelligence Assistant) est un assistant vocal et textuel intégré dans toutes les interfaces KOMOE. Il combine la reconnaissance vocale du navigateur (Web Speech API), la synthèse vocale (SpeechSynthesis), et le modèle **Gemini Flash** de Google avec Function Calling pour accéder aux données en temps réel.

- **Reconnaissance vocale** : Web Speech API (fr-FR) en mode continu — capture permanente, envoi manuel
- **Synthèse vocale** : SpeechSynthesis avec sélection de voix — muet, pause, reprise
- **Context-aware** : Connaît le rôle, la commune, la page courante et le nom de l'utilisateur
- **Function Calling** : `get_commune_projects()`, `get_commune_transactions()`, `get_commune_details()` — données live
- **Sécurité rôle** : Le contexte système interdit à MIA de révéler des données inter-utilisateurs pour les CITOYEN
- **Gestion d'erreurs** : Timeout 15s, quota Gemini (20 req/jour offre gratuite), erreur réseau — messages explicites
- **Animations** : Lottie animations (DotLottieReact) pour idle/thinking/speaking — UX vivante

5.4 Intégration Blockchain Frontend (wagmi + RainbowKit)

Le frontend intègre wagmi v2 + RainbowKit pour la connexion wallet MetaMask côté client (lecture publique des events on-chain). Les transactions d'écriture passent toujours par le backend (mode custodial). Les constantes blockchain sont centralisées dans `lib/constants.ts` et `lib/blockchain.ts` (ABI complet du contrat).

- `wagmi v2 + viem` : Hooks React pour interactions Ethereum
- `RainbowKit` : Modal de connexion wallet (MetaMask, WalletConnect)
- `ethers.js v6` : Interactions bas niveau blockchain
- `react-force-graph-2d` : Visualisation réseau des transactions blockchain
- `@lottiefles/dotlottie-react` : Animations Lottie pour MIA Bot
- `framer-motion` : Animations UI fluides
- `@tanstack/react-query` : Gestion du cache et des fetches asynchrones
- `next-themes` : Mode sombre/clair avec persistance
- `@google/generative-ai` : SDK Gemini pour MIA Bot

6. API REST — Référence Complète

URL de base : **https://[backend-render-url]** en production, **http://localhost:8000** en dev. Toutes les requêtes authentifiées nécessitent le header **Authorization: Bearer {access_token}**.

6.1 Auth & Utilisateurs (/api/auth/)

POST	/api/auth/register/	Inscription libre (CITOYEN uniquement)	Public
POST	/api/auth/login/	Connexion — retourne access + refresh JWT	Public
POST	/api/auth/refresh/	Renouvellement du token d'accès	Public
GET	/api/auth/me/	Profil de l'utilisateur connecté	Auth
PATCH	/api/auth/me/	Mise à jour du profil	Auth
GET	/api/auth/users/	Liste des utilisateurs (filtré par rôle)	Auth
POST	/api/auth/users/	Créer un compte (Admin/Maire)	Auth
GET/PUT/DELETE	/api/auth/users/{id}/	Détail / MAJ / Suppression utilisateur	DGDDL/Maire
GET	/api/auth/users/{id}/engagements/	Historique des engagements	Auth
PATCH	/api/auth/users/{id}/verify-journalist/	Vérifier badge journaliste	DGDDL
PATCH	/api/auth/users/{id}/verify-profession/	Vérifier profession	DGDDL
POST	/api/auth/users/{id}/authorize-blockchain/	Attribuer rôle on-chain	DGDDL
POST	/api/auth/certification/submit/	Soumettre CNI (Sentinelle)	Auth
GET	/api/auth/certification/pending/	Liste certifications en attente	DGDDL/Maire
PATCH	/api/auth/certification/{user_id}/review/	Approuver/rejeter certification	DGDDL/Maire
POST	/api/auth/documents/upload/	Upload document professionnel	Auth

GET	/api/auth/documents/pending/	Documents en attente de vérification	DGDDL
PATCH	/api/auth/documents/{id}/review/ /	Approuver/rejeter document	DGDDL
POST	/api/auth/blockchain/toggle-pause/ se/	Pauser/reprendre contrat	DGDDL

6.2 Communes & Projets (/api/communes/)

GET	/api/communes/	Liste toutes communes actives (avec score)	Public
GET	/api/communes/{id}/	Détail d'une commune	Public
GET	/api/communes/admin/	Liste toutes communes (admin)	DGDDL
POST	/api/communes/admin/	Créer une commune	DGDDL
PATCH	/api/communes/admin/{id}/	Modifier une commune	DGDDL
PATCH	/api/communes/admin/{id}/confirmer-dotation/	Enregistrer hash dotation blockchain	DGDDL
GET	/api/communes/projets/	Liste des projets (filtrable par commune)	Auth
POST	/api/communes/projets/	Créer un projet	Auth
GET	/api/communes/projets/{id}/	Détail projet	Auth
PATCH /DELETE	/api/communes/projets/{id}/	Modifier / supprimer projet	Auth

6.3 Transactions (/api/transactions/)

GET	/api/transactions/	Liste paginée des transactions	Auth
POST	/api/transactions/soumettre/	Créer une dépense (BROUILLON → SOUMIS)	Agent
GET	/api/transactions/{id}/	Détail d'une transaction	Auth
PATCH /DELETE	/api/transactions/{id}/	Modifier / supprimer	Auth
PATCH	/api/transactions/{id}/valider/	Valider sur blockchain (SOUMIS → VALIDE)	Maire
PATCH	/api/transactions/{id}/rejeter/	Rejeter avec motif	Maire
PATCH	/api/transactions/{id}/confirmer-hash/	Confirmer hash soumission blockchain	Agent
GET	/api/transactions/{id}/qr/	QR code de la transaction	Auth
GET	/api/transactions/commune/{id}/	Transactions d'une commune	Auth
POST	/api/transactions/recettes/	Créer une recette	Agent
PATCH	/api/transactions/recettes/{id}/confirmer/	Valider recette on-chain	Maire
GET	/api/transactions/anomalies/	Détection d'anomalies	DGDDL
GET	/api/transactions/commune/{id}/rapport/	Générer rapport PDF	Auth
GET	/api/transactions/open/stats/	Statistiques open data	Public
GET	/api/transactions/open/export/transactions.csv	Export CSV transactions	Public
GET	/api/transactions/open/export/signalements.csv	Export CSV signalements	Public
GET	/api/transactions/notifications/	Liste des notifications	Auth
PATCH	/api/transactions/notifications/read/	Marquer tout comme lu	Auth

GET	/api/transactions/notifications/stream/	Stream SSE temps réel	Auth
------------	---	-----------------------	------

6.4 Signalements & Propositions

GET	/api/transactions/signalements/	Liste des signalements	Auth
POST	/api/transactions/signalements/	Créer un signalement	Auth
GET	/api/transactions/signalements/{id}/	Détail signalement	Auth
POST	/api/transactions/signalements/{id}/voter/	Voter CREDIBLE ou INFONDE	Auth
POST	/api/transactions/signalements/{id}/preuves/	Ajouter une preuve IPFS	Auth
GET/POST	/api/transactions/signalements/{id}/commentaires/	Commentaires du signalement	Auth
PATCH	/api/transactions/signalements/{id}/enquete/lancer/	DGDDL lance l'enquête	DGDDL
POST	/api/transactions/signalements/{id}/enquete/note/	Ajouter note d'enquête	DGDDL
PATCH	/api/transactions/signalements/{id}/enquete/resoudre/	DGDDL résout l'enquête	DGDDL
GET	/api/transactions/propositions/	Liste des propositions citoyennes	Auth
POST	/api/transactions/propositions/	Créer une proposition	Auth
POST	/api/transactions/propositions/{id}/voter/	Voter SOUTIEN ou OPPOSITION	Auth
DELETE	/api/transactions/propositions/{id}/voter/	Retirer son vote	Auth
PATCH	/api/transactions/propositions/{id}/officialiser/	Maire officialise la proposition	Maire
PATCH	/api/transactions/propositions/{id}/cloturer/	Maire clôture le vote	Maire
POST	/api/transactions/propositions/{id}/preuves/	Ajouter une preuve	Auth

GET/ POST	/api/transactions/propositions/ {id}/commentaires/	Commentaires proposition	Auth
--------------	---	--------------------------	------

7. Flux de Données Principaux

Flux 1 — Validation d'une dépense

Étape	Acteur	Action
1	AGENT_FINANCIER	Crée la dépense (POST /soumettre/) → statut BROUILLON en DB
2	AGENT_FINANCIER	Upload justificatif PDF sur IPFS via Pinata → obtient ipfs_hash
3	Backend	Appelle BlockchainService.soumettre_depense() → event DepenseSoumise on-chain
4	Backend	Confirme blockchain_tx_hash_soumission en DB → statut SOUMIS
5	MAIRE	Reçoit notification (SSE/email) — examine la transaction en attente
6	MAIRE	Valide (PATCH /valider/) → Backend appelle BlockchainService.valider_depense()
7	Blockchain	Event DepenseValidee émis avec ipfsHash ancré — preuve publique immuable
8	Backend	Statut → VALIDE, blockchain_tx_hash_validation enregistré, Commune.score recalculé

Flux 2 — Signalement citoyen jusqu'au verdict blockchain

Étape	Acteur	Action
1	CITOYEN	POST /signalements/ — crée le signalement (statut NOUVEAU)
2	Citoyens	POST /signalements/{id}/voter/ — votes CREDIBLE → statut ACTIF
3	Système	Si nb_votes >= 4 → is_prioritaire = True, statut VIRAL — alerte DGDDL
4	DGDDL	PATCH /enquete/lancer/ → BackendService.lancer_enquete() — event EnqueteLancee
5	DGDDL	Investigation, POST /enquete/note/ — journal des actions
6	DGDDL	PATCH /enquete/resoudre/ {resolution: FRAUDE} → resoudre_enquete() on-chain
7	Backend	Si FRAUDE → transaction.statut = FRAUDULEUSE + création correction
8	Blockchain	Event EnqueteResolue avec resolution ancré — verdict public immuable

8. Sécurité

- **Authentification JWT** : Access token 60 min + refresh token 7 jours avec rotation. La clé SECRET_KEY Django est obligatoirement chargée depuis les variables d'environnement.
- **Permissions par rôle** : Chaque endpoint vérifie le rôle via des classes de permission dédiées (IsDGDDL, IsMaire, etc.). DEBUG=False par défaut en production.
- **CORS restreint** : Les origines autorisées sont définies dans CORS_ALLOWED_ORIGINS (variable d'environnement).
- **Clé privée backend** : La clé privée du wallet déployeur DGDDL est stockée en variable d'environnement DEPLOYER_PRIVATE_KEY — jamais dans le code source.
- **Scope commune blockchain** : Le mapping walletCommune[] du contrat empêche un agent/maire d'agir sur une autre commune que la sienne. L'anti-double-validation est assuré par un mapping bytes32 on-chain.
- **Pause d'urgence** : Le contrat intègre un circuit-breaker (Pausable OpenZeppelin) activable par la DGDDL en cas de comportement suspect.
- **Validation IPFS** : Chaque transaction validée on-chain ancre le hash IPFS du justificatif. Un auditeur peut vérifier l'intégrité du fichier en comparant hash blockchain vs hash du fichier téléchargé.
- **Certification Sentinelle** : Les votes sur signalements et propositions nécessitent une certification d'identité vérifiée pour limiter les manipulations de masse.
- **Proxy middleware** : proxy.ts (Next.js) protège les routes côté serveur via cookie komoe_access synchronisé avec le localStorage.
- **Validation mots de passe** : 4 validateurs Django actifs : similarité, longueur minimale, mots courants, tout-numérique.

9. Déploiement

Variables d'environnement Backend (.env)

Variable	Description
SECRET_KEY	Clé secrète Django — forte, aléatoire, jamais en dur
DEBUG	False en production (True uniquement en dev)
DATABASE_URL	URL PostgreSQL (postgresql://user:pass@host:port/dbname)
ALLOWED_HOSTS	Domaines autorisés séparés par virgule
CORS_ALLOWED_ORIGINS	Origines frontend autorisées
POLYGON_AMOY_RPC_URL	URL RPC Alchemy ou Infura pour Polygon Amoy
CONTRACT_ADDRESS	Adresse du contrat BudgetLedger déployé
DEPLOYER_PRIVATE_KEY	Clé privée du wallet admin DGDDL (TRES SENSIBLE)
PINATA_JWT	Token JWT Pinata pour upload IPFS
PINATA_GATEWAY	URL du gateway IPFS Pinata
JWT_ACCESS_TOKEN_LIFETIME_MINUTES	Durée access token (défaut: 60)

Variables d'environnement Frontend (.env.local)

Variable	Description
NEXT_PUBLIC_API_BASE_URL	URL du backend (ex: https://komoe-api.onrender.com)
NEXT_PUBLIC_CONTRACT_ADDRESS	Adresse du contrat Solidity déployé
NEXT_PUBLIC_POLYGON_RPC_URL	URL RPC Polygon Amoy (lecture blockchain frontend)
NEXT_PUBLIC_GEMINI_API_KEY	Clé API Google Gemini pour MIA Bot
NEXT_PUBLIC_REOWN_PROJECT_ID	Project ID Reown/WalletConnect pour RainbowKit

Infrastructure de déploiement

Composant	Service	Détails
Backend API	Render.com (Web Service)	Python 3.13, Gunicorn, WhiteNoise, auto-deploy sur push main
Base de données	Render PostgreSQL	Instance managée, backups automatiques
Frontend	Vercel	Next.js SSR/SSG, CDN mondial, preview deployments sur PR
Fichiers statiques	WhiteNoise (intégré)	Fichiers CSS/JS compressés et mis en cache
Smart contract	Polygon Amoy	Hardhat pour compilation/déploiement, scripts deploy_budget.ts

Stockage documents	Pinata (IPFS)	Gateway publique permanente, PIN management
CI/CD	Git (main branch)	Push sur main → deploy automatique Render + Vercel

Commandes de démarrage

Backend (développement local) :

```
cd backend && python manage.py migrate && python manage.py runserver
```

Backend (production — Profile) :

```
gunicorn config.wsgi:application --bind 0.0.0.0:$PORT
```

Frontend (développement) :

```
npm install && npm run dev
```

Smart contract (compilation + déploiement) :

```
cd contracts && npx hardhat compile && npx hardhat run scripts/deploy_budget.ts --network amoy
```

KOMOE — Plateforme de Transparence Budgétaire

Ce document constitue la documentation technique officielle de la plateforme KOMOE. Pour toute question technique, se référer au dépôt Git du projet ou contacter l'équipe DGDDL.

Document généré automatiquement le 16 May 2026 — Version 1.0

DGDDL · Ministère de l'Intérieur · Côte d'Ivoire · MIABE 2026